XCS224R Assignment 2

Due Sunday, March 8 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at http://xcs231n-scpd.slack.com/
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the environment setup for the coding problems, please refer to the `README.md` file in the assignment directory.
4. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset LaTeX submission. If you wish to typeset your submission and are new to LaTeX, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. For SCPD classes, it is also important that students avoid opening pull requests containing their solution code on the shared assignment repositories. More information regarding the Stanford honor code can be found at https://communitystandards.stanford.edu/policies-and-guidance/honor-code.

Writing Code and Running the Autograder

All your code should be entered into the `src/submission/` directory. When editing files in `src/submission/`, please only make changes between the lines containing `### START CODE HERE ###` and `### END CODE HERE ###`. Do not make changes to files outside the `src/submission/` directory.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- `basic`: These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.

- `hidden`: These unit tests are NOT visible locally. These hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned. These tests will evaluate elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the Anaconda Setup for XCS Courses to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic:  Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic:  Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

This error caused the test to fail.

In this case, start your debugging in line 23 of grader.py.

Remote Execution

Basic and hidden tests are treated the same by the remote autograder, however the output of hidden tests will only appear once you upload your code to GradeScope. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError'>:  {'a': 2, 'b': 1} != None
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError'>:  {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
  File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

1 Overview

In the first part of this assignment, you will experiment with different reward functions in MuJoCo MPC (MJPC), an interactive application for real-time predictive control with MuJoCo. You will design a reward function to achieve a desired agent behavior and observe how different reward functions impact agent behavior. In the second part of the assignment, you will solve a realistic robot task, in which the agent needs to pick up a tool and use it to hammer a nail. You will implement the main parts of an actor-critic algorithm and explore some of the design choices of the algorithm.

Use of GPT/Codex/Copilot/Any AI code assistant/agent: For the sake of deeper understanding on implementing imitation learning methods, assistance from generative models to write code for this homework is prohibited.

You will submit the `submission.py` file together with the `logdir` directory containing the Tensorboard files for the requested two runs only! To do so run the command below from the `src` directory

```
bash collect_submission.sh
```

or you can choose to zip the folder up manually.

2 Impact of Reward Functions

Defining a reward function is often the first step of running any RL algorithm, and pickinga good reward function in practice can be difficult. In this part of the homework, you will get hands-on experience with designing reward functions. Since iterating on reward functions by running RL on them can be slow, we will be using a faster model-based planner to get behavior from the defined reward functions almost instantaneously. The planner does this by leveraging the simulator itself to optimize for behavior with respect to the reward function.

(a) **[5 points (Written, Extra Credit)]**

Design a reward function for the Quadruped task such that the agent walks in a clockwise circle (watch `mujoco_mpc/videos/part1.avi` for an example of the desired behavior). The structure of the reward function for the Quadruped task is:

$$r_t = -w_0 \cdot r_{t,\text{height}} - w_1 \cdot r_{t,\text{pos}}(w_2, w_3) + c$$

where

- $r_{t,\text{height}}$ is the absolute difference between the agent's torso height over its feet and the target height of 1,
- $r_{t,\text{pos}}(w_2, w_3)$ is the ℓ^2 norm of the difference between the agent's torso position and the goal position. The goal moves at each time-step according to the desired *walk speed* w_2 and *walk direction* w_3, and
- c consists of other reward terms for balance, effort, and posture.

You will design the reward function by choosing values for w_0, w_1, w_2, and w_3, which can be any real number. Here is how you can run the Quadruped task with $w_0 = w_1 = w_2 = w_3 = 0$:

```
./build/bin/mjpc --task="Quadruped Flat" --steps=100 \
    --horizon=0.35 --w0=0.0 --w1=0.0 --w2=0.0 --w3=0.0
```

The program will run the simulator for the specified number of time-steps and save a video in the `mujoco_mpc/videos/` directory.

Important: If you are using a headless host, such as the provided Azure Labs VM, make sure you run the following command first before running the simulator, to make the simluator run without a display:

```
xvfb-run -a -s "-screen 0 1400x900x24" bash
```

Parameter	Value
w_0	
w_1	
w_2	
w_3	

Table 1: Parameters of your reward function.

(b) In this next part, you will see how different reward functions impact the agent's behavior in the In-Hand Manipulation task. The structure of the reward function for the Hand task is:

$$r_t = -w_0 \cdot r_{t,\text{cube pos}} - w_1 \cdot r_{t,\text{cube ori}} - w_2 \cdot r_{t,\text{cube vel}} - w_3 \cdot r_{t,\text{actuator}}$$

where

- $r_{t,\text{cube pos}}$ is the ℓ^2 norm of the difference between the cube's position and the hand palm position,
- $r_{t,\text{cube ori}}$ is the ℓ^2 norm of the difference between the cube's orientation and the goal orientation,
- $r_{t,\text{cube vel}}$ is the ℓ^2 norm of the cube's linear velocity, and

- $r_{t,\text{actuator}}$ is the ℓ^2 norm of the control inputs (i.e., the forces applied that cause the robot to move).

For each part below, you will watch the videos located in `mujoco_mpc/videos`. Then, in 1-2 sentences, describe the agent's behavior and why the reward function leads to this behavior.

i. **[4 points (Written)]** Watch `mujoco_mpc/videos/part1bi.avi`, which was generated with the parameters $w_0 = 20$, $w_1 = 3$, $w_2 = 10$, and $w_3 = 0.1$:

```
./build/bin/mjpc --task="Hand" --steps=100 \
    --horizon=2.5 --w0=20.0 --w1=3.0 --w2=10.0 --w3=0.1
```

Describe the agent's behavior and why the reward function leads to this behavior in 1-2 sentences.

ii. **[4 points (Written)]** Watch `mujoco_mpc/videos/part1bii.avi`, which was generated with the parameters $w_0 = 20$, $w_1 = 3$, $w_2 = 10$, and $w_3 = 1$:

```
./build/bin/mjpc --task="Hand" --steps=100 \
    --horizon=0.25 --w0=20.0 --w1=3.0 --w2=10.0 --w3=1.0
```

Describe the agent's behavior and why the reward function leads to this behavior in 1-2 sentences.

iii. **[4 points (Written)]** Watch `mujoco_mpc/videos/part1biii.avi`, which was generated with the parameters $w_0 = 0$, $w_1 = 0$, $w_2 = 0$, and $w_3 = 1$:

```
./build/bin/mjpc --task="Hand" --steps=100 \
    --horizon=2.5 --w0=0.0 --w1=0.0 --w2=0.0 --w3=1.0
```

Hint: An episode terminates when the cube falls out of the hand.

Describe the agent's behavior and why the reward function leads to this behavior in 1-2 sentences.

3 Actor-critic

In this problem we consider a realistic task in which a Sawyer robot needs to grasp a tool and use it to hammer in a nail. The agent controls a 4-DOF Sawyer robot with a continuous action space and receives environment states as observations. Providing dense rewards for real world problems is difficult, so in this environment the agent receives a sparse reward of 1.0 upon fully completing the task and no intermediate rewards. We will implement a general actor-critic algorithm that learns to solve the task with almost 100% success rate. At each step the agent processes observations o_t, which consists of the environment states (e.g., pose of the robot gripper and hammer, etc.) from the last two environment steps. The full agent consists of:

- The actor-critic algorithm utilizes critic networks $Q_{\theta^i}(o_t, a_t)$. This is implemented as the **Critic** class in **ac.py**. We use an ensemble of N different critic networks. Using an ensemble of critic networks can help reduce error in the Q-value estimates.
- Each critic network has a corresponding *target* critic network $\bar{Q}_{\theta^i}(o_t, a_t)$., which is used for computing the Bellman targets for critic updates. We use target critic networks, which are updated more slowly, to have a more stable target for learning.
- The final component of the algorithm is the Actor or policy $\pi_\theta(o_t)$. This is implemented as the **Actor** class in **ac.py**. You should familiarize yourself with these functions and their inputs and outputs. You should not modify any other files besides **ac.py**.

(a) **[6 points (Coding)]**

Optimizing behavior in environments with sparse rewards is difficult due to limited reward supervision. To alleviate this, we provide the agent with 20 successful demonstrations, which we will use to pre-train with behavior cloning. In **ac.py** complete the **bc** function of the **ACAgent** class to train the policy using supervised behavior cloning. That is, given state-action pairs o_t, a_t optimize the loss

$$\mathcal{L}_{\pi_\theta, f_\theta}(o_t, a_t) = -\log \pi_\theta(a_t|o_t)$$

We will also use this to update the actor during later RL training, to improve stability.

(b) **[12 points (Coding)]** In the second part, we will try to improve the performance of the policy with additional fine-tuning with reinforcement learning. Reinforcement learning allows the robot to improve using its own experience, without needing additional demonstrations. Your implementation will be in the **ACAgent** class.

- We begin by implementing the **update_critic** method using the Bellman objective. Consider transitions $(o_t, a_t, r_t, o_{t+1}, \gamma)$ and implement the following steps:
  i. Sample next state actions from the policy $a'_{t+1} \sim \pi_\theta(o_{t+1})$.
  ii. Compute the Bellman targets

  $$y = r_t + \gamma \min\{\bar{Q}_{\theta^i}(o_{t+1}, a'_{t+1}), \bar{Q}_{\theta^j}(o_{t+1}, a'_{t+1})\}$$

  where $\bar{Q}_{\theta^i}$ and $\bar{Q}_{\theta^j}$ are two randomly sampled target critics. We take the minimum over two Q-values to mitigate critic overestimation errors.
  iii. Compute the loss:

  $$\mathcal{L}_{Q_\theta, f_\theta} = \sum_{i=1}^{N} (Q_{\theta^i}(o_t, a_t) - \mathbf{sg}(y))^2$$

  where **sg** stands for the stop gradient operator.
  iv. Take a gradient step with respect to the critic parameters.
  v. Update the target critic parameters using exponential moving average.

  $$\bar{Q}_{\theta^i} = (1 - \tau)\bar{Q}_{\theta^i} + \tau Q_{\theta^i}$$

  By using an exponential moving average, our target critic parameters are updated more slowly than those of the main critic.

- Next, we will improve the policy in the **update_actor** method. Sample an action from the actor $a'_t \sim \pi_\theta(o_t)$ and compute the objective that optimizes the actor to maximize the Q-value estimates from the critics:

$$\mathcal{L}_{\pi_\theta} = -\frac{1}{N}\sum_{i=1}^{N} Q_{\theta^i}(o_t, a'_t)$$

  Take a gradient step on this objective with respect to the policy only.

(c) Once you have finished implementing the code gaps, proceed with the following sub-parts:

i. **[6 points (Coding)]** Run the RL fine-tuning with

```
python train.py agent.num_critics=2 utd=1
```

ii. **[3 points (Written)]** Attach the plot "eval/episode_success" for the above run from Tensorboard. You should achieve a success rate of at least 90% before 100K environment steps.

iii. **[6 points (Coding)]** In the final part, we will explore some optimization parameter choices. The update-to-data (UTD) ratio stands for the number of critic gradient steps we do, with respect to each environment step. So far we have used only 2 critics and UTD of 1. Repeat the previous part with:

```
python train.py agent.num_critics=10 utd=5
```

Here, we perform critic gradient steps 5x more often. However, too many gradient steps can make learning unstable by increasing error in the critic Q-values.

iv. **[5 points (Written)]** Attach the plot "eval/episode_success" for this run and compare it to the previous question. Provide an explanation of why we observe these effects.

Note: this is slower to run due to the higher computational cost and you should expect run time of about 2 hours for 50,000 steps.

Common Bugs

- If your critic loss curves remain flat throughout training, make sure the criticsare being updated.
- If your critic losses become very large, make sure the dimensions of your criticpredictions and targets are correct, and/or tensor broadcasting is done correctly.
- Make sure all critics are updated each iteration, not just those corresponding tothe two target critics that are sampled for computing Bellman targets.